

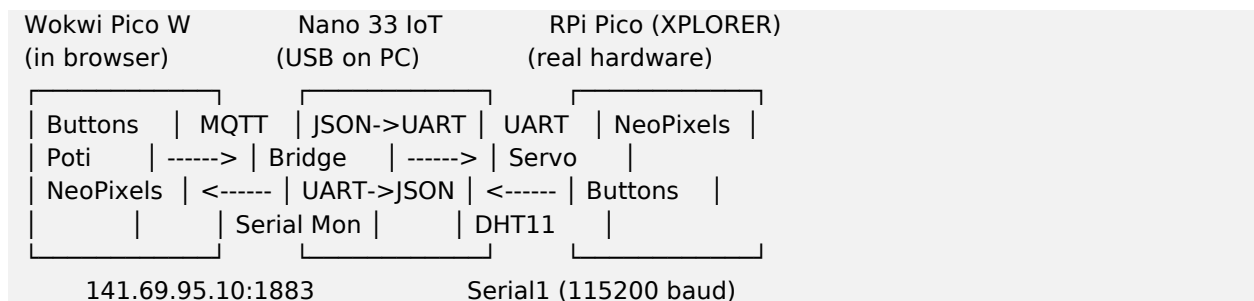
Embedded Lab & Project

Task 4 – UART Bridge to XPLOER Board

This document describes Task 4 of the Embedded Lab & Project course. Task 4 extends the MQTT chain from Task 3 with a physical UART link to a Raspberry Pi Pico on a Joy-IT XPLOER board. Commands flow from the Wokwi simulation through MQTT and UART to control real hardware (NeoPixel LEDs, servo motor) and DHT11 sensor readback. The Wokwi side mirrors the XPLOER LED state on virtual NeoPixels and shows live temperature / humidity on an SSD1306 OLED.

Overview

In Task 4, the communication chain is completed end-to-end. The Wokwi Pico W acts as a remote control panel, the Arduino Nano 33 IoT bridges between MQTT and UART, and the Raspberry Pi Pico on the XPLOER board controls physical hardware.



Property	Details
Duration	2–3 sessions
Environment	Wokwi (Pi Pico W) + Arduino Nano 33 IoT + Joy-IT XPLOER (RPi Pico)
Prerequisites	Lab Task 3 completed
Libraries (Nano)	WiFiNINA, PubSubClient, SimpleJson.h
Libraries (Pico)	Adafruit NeoPixel, DHT sensor library
Libraries (Wokwi)	WiFi.h, PubSubClient, Adafruit NeoPixel, Wire, Adafruit GFX, Adafruit SSD1306, SimpleJson.h
MQTT Broker	141.69.95.10:1883 (no account needed)
UART	Serial1, 115200 baud, plain text KEY:VALUE protocol

From Task 3 to Task 4

In Task 3, the Nano received commands via Serial Monitor and forwarded them via MQTT/JSON to the Wokwi Pico W. In Task 4, the chain is extended: the Nano now also communicates via UART with a real Raspberry Pi Pico on the XPLOER board. The Wokwi Pico W becomes the remote control panel.

Property	Task 3 (MQTT)	Task 4 (UART + XPLOER)
End device	Wokwi Pico W (simulated)	RPi Pico on XPLOER (real)
Transport	MQTT only	MQTT + UART

UART format	–	Plain text KEY:VALUE
Hardware	1 LED, 1 Poti, 1 Button	4 RGB LEDs, Servo, 4 Buttons, DHT11
Nano role	Serial Monitor gateway	MQTT ↔ UART bridge
Wokwi role	LED controller	Remote control panel + mirror
Bidirectional	Status messages	Buttons, sensor data, full status

UART Protocol

Communication between the Arduino Nano 33 IoT and the RPi Pico uses plain text commands over UART (Serial1 at 115200 baud). Each message is terminated with a newline character (n).

Commands (Nano → Pico)

Command	Example	Description
LED:N:NEXT	LED:1:NEXT	Cycle LED N (1–4) to its next color (RED → GREEN → BLUE → YELLOW → CYAN → PURPLE → WHITE → OFF)
SERVO:angle	SERVO:90	Set servo to angle (0–180)
STATUS	STATUS	Request full device status

Supported colors: RED, GREEN, BLUE, WHITE, YELLOW, CYAN, PURPLE, OFF

Responses (Pico → Nano)

Response	Example	Description
ACK:LED:n:COLOR	ACK:LED:1:RED	LED N is now in this color (after Wokwi LED:N:NEXT cmd OR after XPLOER button press)
ACK:SERVO:angle	ACK:SERVO:90	Servo command acknowledged
BTN:name	BTN:TOP	Button press event (TOP/RIGHT/BOTTOM/LEFT)
TEMP:value	TEMP:23.5	Temperature reading (°C)
HUM:value	HUM:45	Humidity reading (%)
STATUS:...	STATUS:LED1=RED,...	Full status report

MQTT Topics

Communication between Wokwi and Nano uses the same JSON+MQTT pattern as Task 3, but with new topics and a cmd/uart field structure.

Topic	Publisher	Subscriber	Content
iem/task4/xplorer/cmd	Pico W (Wokwi)	Nano IoT	JSON with cmd field
iem/task4/xplorer/data	Nano IoT	Pico W (Wokwi)	JSON with uart field

JSON Message Format

Wokwi → Nano (command)

```
{ "token": "iem2026", "source": "pico", "seq": 0, "cmd": "LED:1:NEXT" }
{ "token": "iem2026", "source": "pico", "seq": 1, "cmd": "SERVO:90" }
{ "token": "iem2026", "source": "pico", "seq": 2, "cmd": "STATUS" }
```

The cmd field contains the exact UART command that the Nano forwards to the Pico via Serial1.

Nano → Wokwi (feedback)

```
{ "token": "iem2026", "source": "nano", "seq": 0, "uart": "ACK:LED:1:RED" }
{ "token": "iem2026", "source": "nano", "seq": 1, "uart": "BTN:TOP" }
{ "token": "iem2026", "source": "nano", "seq": 2, "uart": "TEMP:23.5" }
```

The uart field contains the raw UART response from the XPLOER Pico, forwarded as-is.

XPLOER Board Hardware

The Joy-IT XPLOER board provides multiple peripherals controlled by the Raspberry Pi Pico:

Component	GPIO	Details
4x NeoPixel LED	GP1	WS2812 addressable RGB LEDs (single data pin)
Servo Motor	GP7	Standard servo, 0–180 degrees
Button: Top	GP10	Active LOW with internal pull-up
Button: Right	GP11	Active LOW with internal pull-up
Button: Bottom	GP14	Active LOW with internal pull-up
Button: Left	GP15	Active LOW with internal pull-up
DHT11 Sensor	GP22	Temperature (°C) and Humidity (%) sensor
UART TX	GP4	Serial1 TX to Nano RX (D0)
UART RX	GP5	Serial1 RX from Nano TX (D1)

Wiring

Nano 33 IoT ↔ RPi Pico (UART)

Nano 33 IoT RPi Pico (XPLOER)

```
D1 (TX)  —————> GP5 (RX)
D0 (RX)  <———— GP4 (TX)
GND      ————— GND
```

Important: TX and RX must be cross-connected. Both devices must share a common ground.

Pico W Behaviour Specification

This section defines what the Wokwi Pico W must do at the conceptual level, before you start filling in the code stubs. Implementation hints for each function are inside the .ino file as TODO comments. You write firmware for THREE devices in this task: (1) the Wokwi Pico W (file pico_w_task4.ino), (2) the Nano IoT MQTT-UART bridge (file nano_iot_task4.ino), and (3) the XPLOER Pico (file xplorer_pico.ino) — each has its own stub file with its own TODOs. The XPLOER side is the single source of truth for LED state and the actual servo / DHT11 hardware.

Outgoing — Wokwi inputs that send commands to the XPLOER

Trigger	Behaviour
LED Button N (1..4)	On press, publish LED:N:NEXT (one of LED:1:NEXT ... LED:4:NEXT). Do NOT update the local NeoPixel directly — wait for the ACK:LED:N:COLOR response and let processUARTFeedback() apply the new color, so XPLOER stays the single source of truth.
Potentiometer	Map ADC value (0–4095) to angle (0–180). Throttle to one read per 100 ms; publish SERVO:<angle> only when angle changed by more than $\pm 5^\circ$ since the last publish.

Incoming — UART feedback from XPLOER, mirrored on Wokwi

Trigger	Behaviour
ACK:LED:n:COLOR	Update virtual NeoPixel N (1..4) to the given color and call pixels.show(). This message arrives both after Wokwi LED:N:NEXT commands AND after physical XPLOER button presses (XPLOER is the single source of truth for LED state).
ACK:SERVO:angle	Update currentServoAngle and log to Serial. There is no local servo on the Wokwi side.
TEMP:value, HUM:value	Parse the value (substring(5).toFloat() for TEMP, substring(4).toInt() for HUM), cache it in lastTempC / lastHumPct, log to Serial AND update the SSD1306 OLED via renderDisplay() so the latest reading stays on screen.
STATUS:LED1=...,SERVO=...	Comma-separated key=value list with full XPLOER state. Use the provided parseStatusReport() helper to update all virtual NeoPixels and the mirrored servo angle in one shot.

XPLOER Pico Behaviour Specification

This section defines what the XPLOER-side firmware (file xplorer_pico.ino) must do. The XPLOER owns the real LED colors, the real servo angle, and the DHT11 sensor — every state change here is broadcast back to the Wokwi side via the Nano bridge so the Wokwi mirror stays in sync. Implementation hints are inside the .ino file as TODO comments.

UART commands the XPLOER must accept (from Nano bridge)

Command	Behaviour
LED:N:NEXT (N=1..4)	Advance LED N to its next color in the sequence (RED → GREEN → BLUE → YELLOW → CYAN → PURPLE → WHITE → OFF). Drive the physical NeoPixel and reply ACK:LED:N:COLOR with the

	new color name.
SERVO:angle	Constrain angle to [0..180], call servo1.write(angle), update the servoAngle state variable, reply ACK:SERVO:angle.
STATUS	Reply with one STATUS:... line containing all current LED colors, the servo angle, and the most recent TEMP/HUM readings (use the provided sendStatusReport() helper).

Events the XPLOER must emit (to Nano bridge)

Event	Behaviour
Physical button TOP / RIGHT / BOTTOM / LEFT pressed	Treat as the LED-cycle trigger for that side: TOP → LED 1, RIGHT → LED 2, BOTTOM → LED 3, LEFT → LED 4. Call advanceLedColor(N) — it advances the color, drives the LED AND emits ACK:LED:N:COLOR for you. Use BUTTON_DEBOUNCE (50 ms) and falling-edge detection.
DHT11 reading (every DHT_READ_INTERVAL ms)	Read humidity and temperature; if BOTH are valid (isnan check), update lastTemperature / lastHumidity and emit TEMP:value plus HUM:value over UART. On NaN, log to USB Serial and skip this cycle.
LED color changed (any reason)	advanceLedColor() centralises this: it drives the NeoPixel AND emits ACK:LED:N:COLOR. Both the LED:N:NEXT command handler and the button handler call this same helper, keeping XPLOER as the single source of truth.

Task Steps

Step 1: Set up the RPi Pico on XPLOER Board

See also separate document

1. Arduino IDE: Install the board package "Raspberry Pi Pico/RP2040" by Earle Philhower (arduino-pico).
2. Install libraries: Adafruit NeoPixel, DHT sensor library.
3. Copy the starter stub (xplorer_pico.ino) into your sketch folder.
4. Select board: "Raspberry Pi Pico" and the correct COM port.
5. Verify that all peripherals are enabled on the XPLOER board (check the DIP switches).
6. Upload and open Serial Monitor (115200 baud) – you should see the startup banner.

Step 2: First End-to-End Test via Nano Serial Monitor

Note: The Nano bridge firmware blocks in mqttReconnect() until the broker is reachable, so this step is not really “local” — WiFi and the lab MQTT broker (141.69.95.10) must already be available before the Nano enters its main loop. If the Nano stays stuck on “Connecting to MQTT...”, fix the network first; UART will not respond until then.

7. Connect the Nano 33 IoT to the Pico via UART (see Wiring section).
8. Open the Nano Serial Monitor (115200 baud, Newline line ending).
9. Copy the Nano bridge stub (nano_iot_task4.ino) and SimpleJson.h to your sketch folder.
10. Upload the bridge code to the Nano.

11. Type LED:1:NEXT in the Nano Serial Monitor – LED 1 on the XPLOER should advance to the next color (RED first).
12. Type SERVO:90 – the servo should move to 90 degrees.
13. Press a button on the XPLOER – you should see BTN:TOP (etc.) in the Serial Monitor.

Step 3: Connect Wokwi via MQTT

14. Start the Wokwi Private IoT Gateway on your desktop. The lab MQTT broker (141.69.95.10) is on the HS-internal network, so the free public Wokwi simulator cannot reach it — only the Private Gateway (paid Wokwi feature, configured on the lab account) tunnels Wokwi traffic into the campus network. Use the lab login.
15. Create a new Wokwi project: Raspberry Pi Pico W.
16. Copy pico_w_task4.ino, SimpleJson.h, and diagram.json into the Wokwi project.
17. Add the PubSubClient, Adafruit NeoPixel, Adafruit GFX and Adafruit SSD1306 libraries via the Wokwi Library Manager.
18. Start the simulation – the Pico W should connect to WiFi and MQTT.
19. Click buttons in Wokwi – commands should flow through to the XPLOER board.
20. Press buttons on the XPLOER – feedback should appear in Wokwi.

Step 4: Full Chain Test

Test the complete bidirectional chain:

Wokwi: Press LED 1 button repeatedly → XPLOER LED 1 cycles through colors (RED → GREEN → BLUE → ...)

Wokwi: Press LED 2/3/4 buttons → XPLOER LED 2/3/4 cycle independently

Each press → MQTT LED:N:NEXT cmd → XPLOER advances → ACK:LED:N:COLOR back → Wokwi NeoPixel mirror updates

Wokwi: Turn potentiometer -> XPLOER servo moves

XPLOER: Press button N → XPLOER LED N cycles → Wokwi virtual LED N mirrors the new color

Type 'status' in Wokwi Serial Monitor → STATUS:... reply parsed → Wokwi mirrors full XPLOER state at once. Live DHT11 readings appear on the SSD1306 OLED next to the Pico W.

-> Wokwi NeoPixels mirror XPLOER LED state (servo is fire-and-forget from Wokwi)

Deliverable

✓ **Deliverable:** The full end-to-end chain works bidirectionally: Wokwi LED button → MQTT LED:N:NEXT → Nano → UART → XPLOER cycles LED N and replies ACK:LED:N:COLOR → Wokwi mirrors. Pot → SERVO:angle → real servo. XPLOER buttons / DHT11 / STATUS replies flow back the same way. Both sides stay in sync via XPLOER as single source of truth. Robustness mechanisms (token, sequence numbers, watchdog) are implemented. Code is commented.

Hints

UART debugging: Open two Serial Monitors – one for the Nano (USB) and one for the Pico (USB). This lets you see both sides of the UART conversation simultaneously.

NeoPixel library (Pico): The arduino-pico core supports Adafruit NeoPixel natively. Use `pixels.begin()`, `pixels.setPixelColor(n, r, g, b)`, `pixels.show()`.

Serial1 on Pico: The arduino-pico core allows remapping UART pins with `Serial1.setTX(4)` and `Serial1.setRX(5)` before calling `Serial1.begin()`.

DHT11 readings: The DHT11 sensor can return NaN if read too quickly. Always check with `isnan()` before using the values. Minimum interval between reads: 2 seconds.

String parsing: Use `indexOf(':')` and `substring()` to split UART commands. For example: `cmd.indexOf(':')` gives the position of the first colon.

XPLOER 5V Power: The servo and the four NeoPixel RGB LEDs need a 5V supply that the Pico USB cannot provide. Use the second USB-C port located underneath the Pico on the XPLOER board to power them. If you skip this, the servo will jitter and the LEDs will flicker under load.

Wokwi simulation: The Wokwi diagram does NOT include a virtual servo. The servo only exists physically on the XPLOER. The Wokwi side acts as a remote control: the potentiometer reads an angle and sends a `SERVO:angle` command via MQTT. The Pico does not need the Servo Arduino library, since it never drives a local servo.